

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.axes_grid1 import make_axes_locatable
from matplotlib.animation import FuncAnimation
import cv2 as cv
from kwave.kgrid import kWaveGrid
from kwave.kmedium import kWaveMedium
from kwave.ksource import kSource
from kwave.ksensor import kSensor
from kwave.kspaceFirstOrder2D import kspaceFirstOrder2D
from decimal import Decimal

plt.style.use('default')

```

```

rndseed = 42
np.random.seed(rndseed)

# define the computational grid
dx = 10e-6 # grid point spacing in the x and y direction [m]
dy = 10e-6 # grid point spacing in the x and y direction [m]
Nx = 1024*2 # number of grid points in the x direction
Ny = 1024 # number of grid points in the y direction

grid = kWaveGrid([Nx, Ny], [dx, dy])
X , Y = np.meshgrid(grid.x_vec, grid.y_vec , indexing='ij')
grid.makeTime(c=1540, t_end=2e-5, cfl=0.1)

```

```

(array([[0.00000000e+00, 6.49350649e-10, 1.29870130e-09, ...,
        1.99987013e-05, 1.99993506e-05, 2.00000000e-05]]),
 np.float64(6.493506493506495e-10))

```

```

class BreastMap:
    def __init__(self, Nx, Ny, dx, dy, rndseed, max_num_tumors=3):
        self.Nx = Nx
        self.Ny = Ny
        self.dx = dx
        self.dy = dy
        self.max_num_tumors = max_num_tumors # Configuration store
        np.random.seed(rndseed)
        # Initialize empty maps
        self.sos_map = np.zeros((Nx, Ny))
        self.rho_map = np.zeros((Nx, Ny))
        # Internal tracking for layers
        self.epi_thickness = 0

```

```

self.total_skin = 0

def microns2pixels(self, thickness):
    """
    Converts physical thickness from micrometers (μm) to grid p

    Args:
        thickness (float): The thickness in micrometers (e.g.,

    Returns:
        int: The equivalent number of pixels based on the verti
    """
    return int(Decimal(str(thickness)) * Decimal('1e-6') / Deci

def addGel(self, var=2.5):
    """
    Simulates a Propylene Glycol coupling gel layer at the begi

    Args:
        var (float): Standard deviation for the Gaussian noise
    """
    gel_thickness = 1

    # Speed-of-Sound (SOS) for Propylene Glycol: sampled from a
    sos_val = np.random.uniform(1467.805 , 1541.057)

    # Density (Rho) for Propylene Glycol: sampled from range [1
    rho_val = np.random.uniform(1036 , 1040)

    # Apply values to the first 'gel_thickness' columns with ad
    self.sos_map[:, :gel_thickness] = np.random.normal(sos_val,
    self.rho_map[:, :gel_thickness] = np.random.normal(rho_val,

    self.gel_thickness = gel_thickness
    return self

def addEpidermis(self, var=2.5):
    """
    Simulates the epidermis layer based on biological tissue pr
    The layer is placed immediately after the coupling gel.

    Args:
        var (float): Internal variance (porosity) to simulate t
    """
    # 1. Randomize layer thickness and acoustic properties from
    # Thickness range: ~84.5 to 156.7 um (Ref: Measurement of E
    low_h = self.microns2pixels(120.6 - 36.1)
    high_h = self.microns2pixels(120.6 + 36.1)

```

```

epiderm_thickness = np.random.randint(low_h, high_h)

# Sound Speed (c) and Density (rho) randomized within exper
sos_val = np.random.uniform(1531.9, 1735.5) # Range: 1633
rho_val = np.random.uniform(1095.11, 1122.89) # Range: 1109

# 2. Define layer boundaries (starts after the gel layer)
start = self.gel_thickness
end = start + epiderm_thickness

# 3. Populate maps with Gaussian noise to simulate realisti
self.sos_map[:, start:end] = np.random.normal(sos_val, var,
self.rho_map[:, start:end] = np.random.normal(rho_val, var,

self.epi_thickness = epiderm_thickness
return self

def addDermis(self, var=2.5):
    """
    Simulates the dermis layer by placing it after the epidermi
    Physical properties and thickness are derived from clinical

    Args:
        var (float): Standard deviation for Gaussian noise to s
    """
    # 1. Randomize layer thickness based on Korean adult skin s
    # The thickness is converted from microns to pixel indices.
    low_h = self.microns2pixels(1337.9 - 332.3)
    high_h = self.microns2pixels(1337.9 + 332.3)
    derm_thickness = np.random.randint(low_h, high_h)

    # 2. Randomize acoustic properties (Speed of Sound and Dens
    # SoS range [1568.9, 1680.5] m/s | Rho range [1095.11, 1122
    sos_val = np.random.uniform(1568.9, 1680.5)
    rho_val = np.random.uniform(1095.11, 1122.89)

    # 3. Calculate spatial boundaries
    # Note: 'start' begins exactly where the epidermis ends
    start = self.epi_thickness
    end = start + derm_thickness

    # 4. Update property maps with normal distribution to refle
    self.sos_map[:, start:end] = np.random.normal(sos_val, var,
    self.rho_map[:, start:end] = np.random.normal(rho_val, var,

    # 5. Store the total depth of the skin (gel + epidermis + d
    self.total_skin = end

```

```

        return self

def addSubstance(self, var=2.5):
    """
    Fills the remaining depth of the maps with background tissue
    This function completes the map from the end of the skin layer

    Args:
        var (float): Standard deviation for Gaussian noise to noise

    """
    # 1. Randomize background properties within typical soft tissue
    # SoS: 1450–1540 m/s | Rho: 950–1020 kg/m^3
    bg_sos_val = np.random.uniform(1450, 1540)
    bg_rho_val = np.random.uniform(950, 1020)

    # 2. Calculate the thickness of the remaining space in the
    remaining_depth = self.Ny - self.total_skin

    # 3. Fill the maps from the total_skin boundary to the bottom
    # Uses a normal distribution to simulate the stochastic nature
    self.sos_map[:, self.total_skin:] = np.random.normal(bg_sos_val, var,
        self.sos_map[:, self.total_skin:].shape)
    self.rho_map[:, self.total_skin:] = np.random.normal(bg_rho_val, var,
        self.rho_map[:, self.total_skin:].shape)

    return self

def addTumors(self):
    """
    Randomly populates the tissue with elliptical lesions (tumors)
    'Deep Learning for Ultrasound Speed-of-Sound Reconstruction'

    This method simulates diverse clinical cases by varying sizes
    and acoustic properties (benign/malignant-like variations).
    """
    # Create a coordinate grid for mask calculation
    x = np.linspace(0, (self.Nx-1)*self.dx, self.Nx)
    y = np.linspace(0, (self.Ny-1)*self.dy, self.Ny)
    X, Y = np.meshgrid(x, y, indexing='ij')

    # Determine the number of tumors to generate (up to max_num_tumors)
    num_tumors = np.random.randint(1, self.max_num_tumors + 1)

    for _ in range(num_tumors):
        # 1. Randomized Positioning: Ensure tumors are placed within the
        t_cx = np.random.randint(int(0.1*self.Nx), int(0.9*self.Nx))
        t_cy = np.random.randint(self.total_skin + 10, int(0.9*self.Ny))

        # 2. Randomized Geometry: Define radii and rotation angles
        t_rx = np.random.uniform(10, 50) * self.dx * 10

```

```

t_ry = np.random.uniform(10, 30) * self.dx * 10
angle = np.random.uniform(0, np.pi)

# 3. Coordinate Transformation: Apply rotation and translation
X_rot = (X - t_cx) * np.cos(angle) + (Y - t_cy) * np.sin(angle)
Y_rot = -(X - t_cx) * np.sin(angle) + (Y - t_cy) * np.cos(angle)

# Define the elliptical mask
mask = (X_rot**2 / t_rx**2 + Y_rot**2 / t_ry**2) <= 1

# 4. Acoustic Property Assignment (Diverse Tissue Simulation)
# 70% chance: High SOS/Rho (Typical of many solid lesions)
if np.random.rand() > 0.3:
    t_sos, t_rho = np.random.uniform(1550, 1700), np.random.uniform(1000, 1200)
# 30% chance: Low SOS/Rho (Simulating fluid-filled cysts)
else:
    t_sos, t_rho = np.random.uniform(1300, 1450), np.random.uniform(100, 200)

# 5. Apply properties to maps with internal heterogeneity
self.sos_map[mask] = t_sos + np.random.normal(0, 1.5, mask.size)
self.rho_map[mask] = t_rho + np.random.normal(0, 1.5, mask.size)

return self

def get_sos(self):
    return self.sos_map.copy()

def get_rho(self):
    return self.rho_map.copy()

```

```

sos_maps, den_maps = [], []
num_data = 3
m = 0
for i in range(40 , 40+ num_data):
    # 1. Create the object
    m = BreastMap(Nx, Ny, dx, dy, rndseed = i)

    # 2. Pipeline (Method Chaining)
    m.addGel()
    m.addEpidermis()
    m.addDermis()
    m.addSubstance()
    m.addTumors()

    # 3. Collect results
    sos_maps.append(m.get_sos())
    den_maps.append(m.get_rho())

```

```

# Assuming num_data is the number of maps generated
fig, axes = plt.subplots(2, num_data, figsize=(20, 10))

for i in range(num_data):
    # --- Plot SoS Map ---
    # Fixed extent: [0, Nx, 0, Ny] ensures Depth 0 is at the top and
    im1 = axes[0, i].imshow(sos_maps[i].T[::-1 , :], extent=[0, Nx,
    axes[0, i].set_title(f'SoS Map {i+1}')
    axes[0, i].set_xlabel('Lateral (grid points)')
    axes[0, i].set_ylabel('Depth (grid points)')

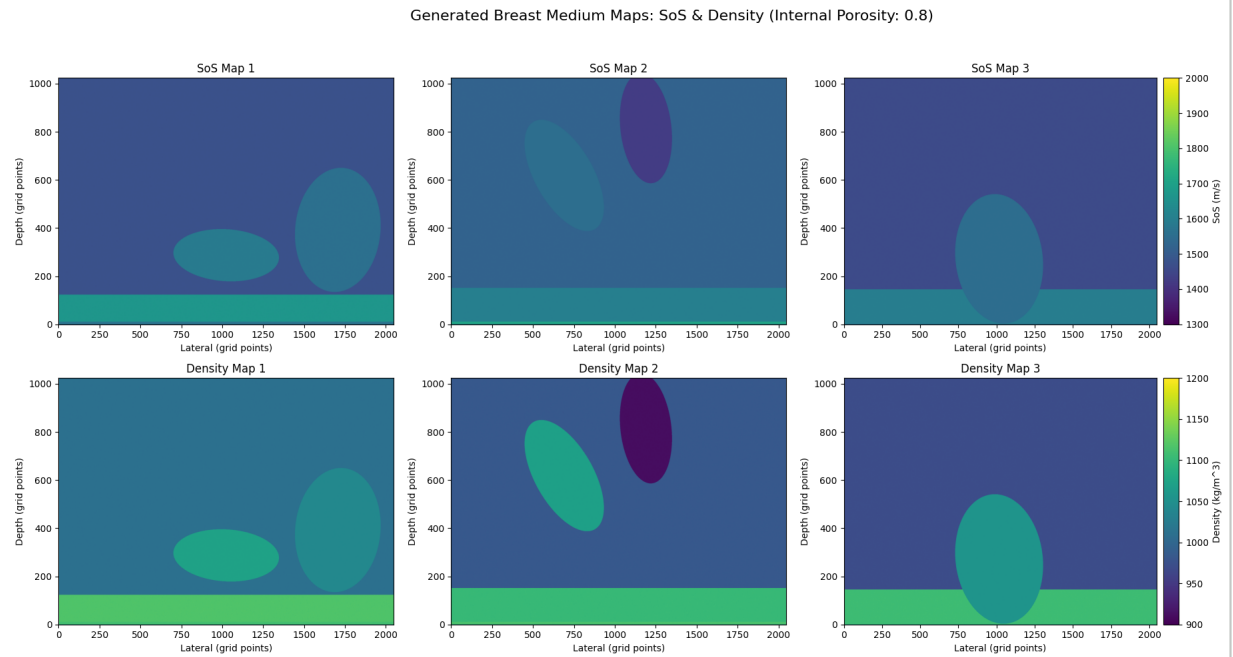
    # Corrected colorbar logic to use num_data index
    if i == num_data - 1:
        divider1 = make_axes_locatable(axes[0, i])
        cax1 = divider1.append_axes("right", size="5%", pad=0.1)
        fig.colorbar(im1, cax=cax1, label='SoS (m/s)')

    # --- Plot Density Map ---
    im2 = axes[1, i].imshow(den_maps[i].T[::-1 , :], extent=[0, Nx,
    axes[1, i].set_title(f'Density Map {i+1}')
    axes[1, i].set_xlabel('Lateral (grid points)')
    axes[1, i].set_ylabel('Depth (grid points)')

    # Corrected colorbar logic for density row
    if i == num_data - 1:
        divider2 = make_axes_locatable(axes[1, i])
        cax2 = divider2.append_axes("right", size="5%", pad=0.1)
        fig.colorbar(im2, cax=cax2, label='Density (kg/m^3)')

```

```
plt.suptitle('Generated Breast Medium Maps: SoS & Density (Internal
plt.tight_layout(rect=[0, 0, 0.92, 0.95])
plt.show())
```



```
medium= []
for i in range(num_data):
    medium.append(kWaveMedium(sound_speed=sos_maps[i], density=density_maps[i]))
```

```

"""#defining the source and sensor (sonographic setup) :
sensor_length = 30 #300 \mu m
pitch = 2 #30 \mu m
null_sider_areas = np.zeros((int(Nx/4), Ny), dtype=bool)
imaging_area = np.zeros((int(Nx/2) , Ny) , dtype=bool)
# Create repeating pattern: sensor_length x True + pitch x False =
pattern_length = sensor_length + pitch
pattern = np.array([True]*sensor_length + [False]*pitch)

# Tile the pattern to cover the entire row
tiled_pattern = np.tile(pattern, Ny // pattern_length + 1)[:Ny]

# Apply pattern to the last row
imaging_area[:, -1] = tiled_pattern

#Generate the entire map
s_mask = np.vstack((null_sider_areas , imaging_area , null_sider_are
#ploting the source pos in the map
plt.imshow(s_mask.T, cmap='gray')
plt.title('Source and Sensor Mask (Line Source at bottom Edge)')
plt.xlabel('Lateral (grid points)')
plt.ylabel('Depth (grid points)')
plt.show()

#defining the sensors :
sensor = kSensor(mask=s_mask, record=['p'])
"""

```

```

<>:2: SyntaxWarning: invalid escape sequence '\m'
<>:2: SyntaxWarning: invalid escape sequence '\m'
/tmp/ipykernel_2005/4268354185.py:2: SyntaxWarning: invalid escape s
    sensor_length = 30 #300 \mu m
'#defining the source and sensor (sonographic setup) : \nsensor_le
ngth = 30 #300 \\mu m \npitch = 2 #30 \\mu m \nnull_sider_areas = n
p.zeros((int(Nx/4), Ny), dtype=bool)\nimaging_area = np.zeros((int(
Nx/2) , Ny) , dtype=bool)\n# Create repeating pattern: sensor_lengt
h x True + pitch x False = period 32\npattern_length = sensor_lengt
h + pitch\npattern = np.array([True]*sensor_length + [False]*pitch)
\n\n# Tile the pattern to cover the entire row\ntiled_pattern = np.

```



```
def microns2grid(length):
    dx = Decimal('10e-6')
    return int(Decimal(str(length)) * Decimal('1e-6') / dx)
```

```
sensor_length = 300 #300 \mu m
```

```
dx = Decimal('10e-6')
int(Decimal(str(sensor_length)) * Decimal('1e-6') / dx)
```

```
30
```

```
numbers_of_sensors = 32;
sensor_length = 30 #300 \mu m
pitch = 2 #30 \mu m
pattern_length = sensor_length + pitch
pattern = np.array([True]*sensor_length + [False]*pitch)

# Tile the pattern to cover the entire row
imaging_area_last_row = np.tile(pattern, numbers_of_sensors)

imaging_area = np.zeros((int(Nx-len(imaging_area_last_row)) , Ny) ,
imaging_area[:, -1] = imaging_area_last_row

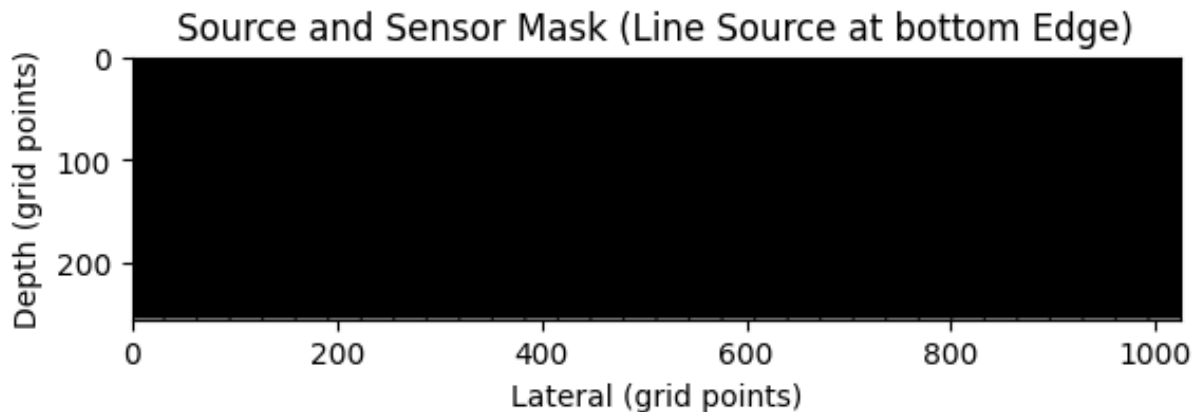
sider_area = np.zeros((int((Nx-len(imaging_area_last_row))/2) , Ny)

s_mask = np.vstack((sider_area , imaging_area ,sider_area))
#defining the sensors :
sensor = kSensor(mask=s_mask, record=['p'])
```

- ✓ plotting the cropped map to check the source pos in the map

```
cropped_map = s_mask[Nx//2-Nx-len(imaging_area_last_row)//2:Nx//2+1

#ploting the cropped map to check the source pos in the map
plt.imshow(cropped_map.T, cmap='gray')
plt.title('Source and Sensor Mask (Line Source at bottom Edge)')
plt.xlabel('Lateral (grid points)')
plt.ylabel('Depth (grid points)')
plt.show()
```



```
# 1. Create the base pulse (Tone Burst)
source = kSource()
amplitude = 1
num_cycles = 3
f0 = 5e6 # 5 MHz center frequency
t_pulse = np.arange(0, num_cycles / f0, grid.dt)
# Gaussian windowed sine wave
pulse_signal = amplitude * np.sin(2 * np.pi * f0 * t_pulse) * np.exp

# Pad with zeros to match simulation time
source_signal = np.zeros(grid.Nt)
source_signal[:len(pulse_signal)] = pulse_signal
# # Define focal point (e.g., 15mm deep in the center)
# x_focus = Nx*dx//2 + 15e-3
# y_focus = Ny*dy//2

# # Get coordinates of all active source elements
# element_idx = np.where(s_mask)
# x_coords = X[element_idx]
# y_coords = Y[element_idx]

# # Calculate distances to focus
# distances = np.sqrt((x_coords - x_focus)**2 + (y_coords - y_focus)**2)

# # Convert distance to time delays (relative to the furthest element)
# sound_speed = 1540 # [m/s]
```

```
# time_delays = (np.max(distances) - distances) / sound_speed
# delay_offsets = np.round(time_delays / grid.dt).astype(int)

source.p_mask = s_mask.astype(float)
# source.p should have shape (num_source_elements, num_time_steps)
num_source_elements = int(np.sum(s_mask))
source.p = np.tile(source_signal.reshape(1, -1), (num_source_element

# #Create the Apodization Weights (Hann Window)
# # We normalize the position of elements from 0 to 1 across the wid
# y_min, y_max = np.min(y_coords), np.max(y_coords)
# normalized_y = (y_coords - y_min) / (y_max - y_min)

# # Calculate Hann weights: 0.5 * (1 - cos(2 * pi * n))
# apodization_weights = 0.5 * (1 - np.cos(2 * np.pi * normalized_y))

# Apply weights to source.p
# Assuming you already have your source.p matrix (num_elements, num_
# We multiply each row (element) by its corresponding weight
#source.p = source.p * apodization_weights.reshape(-1, 1)
```

```
from kwave.options.simulation_options import SimulationOptions
from kwave.options.simulation_execution_options import SimulationExe
sim_options = SimulationOptions(pml_inside=False, save_to_disk=True)
exec_options = SimulationExecutionOptions(is_gpu_simulation=True, bac

sensor_data = kspaceFirstOrder2D(grid, source, sensor, medium[2], si
```

Reading simulation configuration.		Done
Selected GPU device id:		0
GPU device name:		Tesla T4
Number of CPU threads:		2
Processor name:	Intel(R) Xeon(R) CPU @ 2.00GHz	
Simulation details		
Domain dimensions:		2088 x 1064
Medium type:		2D
Simulation time steps:		30801
Initialization		
Memory allocation:		Done
Data loading:		Done
Elapsed time:		0.10s
FFT plans creation:		Done
Pre-processing phase:		Done

Elapsed time:			0.50s
Computational resources			
Current host memory in use:			1385MB
Current device memory in use:			422MB
Expected output file size:			112MB
Simulation			
Progress	Elapsed time	Time to go	Est. finish time
0%	0.004s	68.494s	01/06/26 12:27:32
5%	6.599s	125.223s	01/06/26 12:28:35
10%	13.196s	118.678s	01/06/26 12:28:35
15%	19.803s	112.165s	01/06/26 12:28:35
20%	26.421s	105.644s	01/06/26 12:28:35
25%	33.044s	99.103s	01/06/26 12:28:36
30%	39.678s	92.557s	01/06/26 12:28:35
35%	46.421s	86.190s	01/06/26 12:28:36
40%	53.071s	79.590s	01/06/26 12:28:36
45%	59.735s	72.995s	01/06/26 12:28:35
50%	66.405s	66.392s	01/06/26 12:28:36
55%	73.200s	59.880s	01/06/26 12:28:36
60%	79.888s	53.249s	01/06/26 12:28:37
65%	86.585s	46.613s	01/06/26 12:28:36
70%	93.301s	39.978s	01/06/26 12:28:36
75%	100.029s	33.336s	01/06/26 12:28:37
80%	106.771s	26.686s	01/06/26 12:28:36
85%	113.523s	20.028s	01/06/26 12:28:37
90%	120.281s	13.359s	01/06/26 12:28:37
95%	127.042s	6.682s	01/06/26 12:28:37
Elapsed time:			133.84s
Sampled data post-processing:			Done
Elapsed time:			0.00s
Summary			

```
sensor_data['p'] != np.nan
```

```
sensor_data['p'] != np.nan
```

```
array([[ True,  True,  True, ...,  True,  True,  True],
       [ True,  True,  True, ...,  True,  True,  True],
       [ True,  True,  True, ...,  True,  True,  True],
       ...,
       [ True,  True,  True, ...,  True,  True,  True],
       [ True,  True,  True, ...,  True,  True,  True],
       [ True,  True,  True, ...,  True,  True,  True]], shape=
(6161, 64))
```

```
sensor_length = 30
```

```
sensor_elements_data = sensor_data['p']
```

```
# 1. Determine the number of complete channels we can form
num_channels = sensor_elements_data.shape[1] // sensor_length
valid_cols = num_channels * sensor_length
```

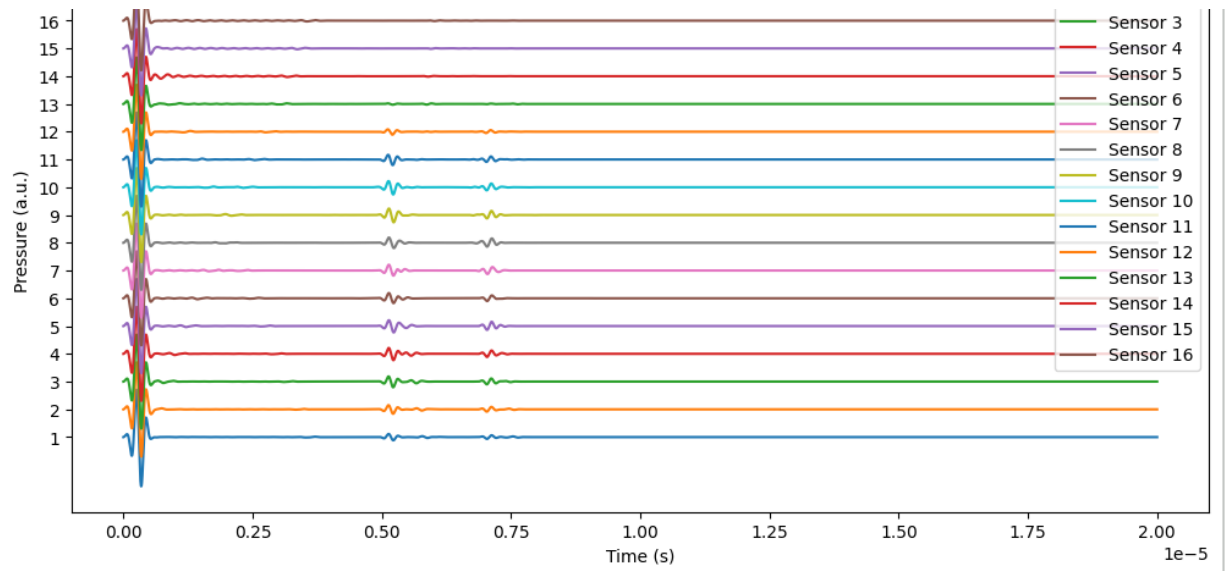
```
# 2. Slice to drop any trailing columns that don't make a full chan
trimmed_data = sensor_elements_data[:, :valid_cols]
```

```
# 3. Reshape and sum along the sensor_length dimension, then average
RF_data = trimmed_data.reshape(trimmed_data.shape[0], -1, sensor_le
```

```
# sensor_data['p'][time,sensor_element] gives the pressure at a spe
#plotting the received signals at the sensors in parallel channels :
"""plt.figure(figsize=(12, 6))
for i in range(RF_data.shape[1]):
    plt.plot(grid.t_array.flatten(), RF_data[:, i] + i*0.5, label=f
plt.title('Received Signals at Sensors (Time vs Pressure)')
plt.xlabel('Time (s)')
plt.ylabel('Pressure (a.u.)')
plt.yticks(np.arange(0, RF_data.shape[1]*0.5, 0.5), [f'Sensor {i+1}
plt.legend(loc='upper right')
plt.show()"""
plt.figure(figsize=(12, 6))
for i in range(RF_data.shape[1]):
    plt.plot(grid.t_array.flatten(), RF_data[:, i] + i*0.5, label=
plt.title('Received Signals at Sensors (Time vs Pressure)')
plt.xlabel('Time (s)')
plt.ylabel('Pressure (a.u.)')
plt.yticks(np.arange(0, RF_data.shape[1]*0.5, 0.5), [i+1 for i in r
plt.legend(loc='upper right')
plt.show()
```

Received Signals at Sensors (Time vs Pressure)

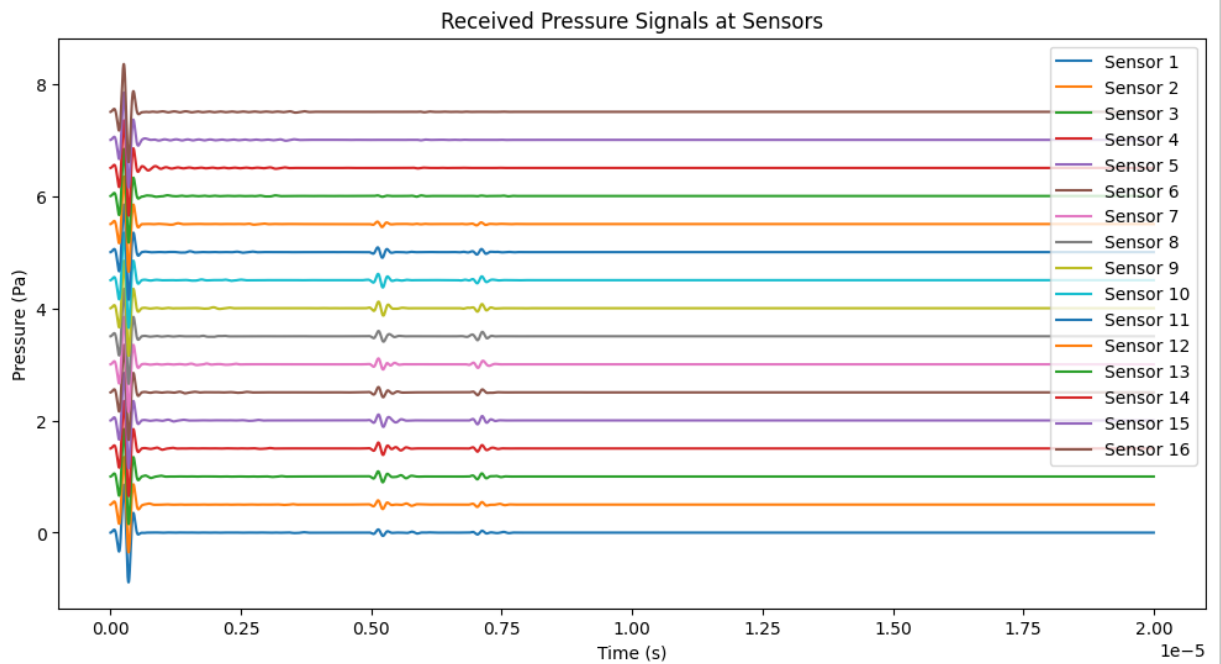




```

plt.figure(figsize=(12, 6))
for i in range(RF_data.shape[1]):
    # Use .flatten() to convert (1, Nt) to (Nt,)
    plt.plot(grid.t_array.flatten(), RF_data[:, i] + i*0.5, label=f
plt.title('Received Pressure Signals at Sensors')
plt.xlabel('Time (s)')
plt.ylabel('Pressure (Pa)')
plt.legend()
plt.show()

```



Start coding or [generate](#) with AI.

